Chapter 17 worked solutions • Distributed systems

OS17-E01 • Speedup and fault domains

1. T6=30+90/6+6=51 s;speedup120/51≈2.353. Ignoring coordination, the bound is120/30=4. Nodes remove neither serial nor coordination work.
2. Independent failure=.02^3=.000008=8×10^{-6}. Shared-rack failures can correlate, invalidating multiplication. Unavailable metadata can also prevent serving surviving copies. The result is not the whole-service SLO.

OS17-E02 • Moving data or computation

1. Transfer512/64+.010=8.010 s before local compute. Remote=.200+2/64+.010=.24125 s. Include each stated component rather than labeling network time total application time. MiB units match.
2. No. Full running state includes PC, registers, memory, resources, and compatible ABI. Starting another program is placement/remote execution.

OS17-E03 • BDP and windows

1. BDP=200 • 10^6 • .05=10000000 bits=1250000 B≈1220.70 KiB. Window bound=131072 • 8/.05=20.97152 Mbit/s, below200.
2. No. Unchanged window/RTT preserves20.97152 Mbit/s. Remove that limit and other bottlenecks before expecting higher link utilization. Convert bytes to bits;RTT is not one-way delay.

OS17-E04 • DNS cache and identity

1. At150,.20 can remain with TTL150. After300, successful resolution obtains the then-current value;failed networking does not establish a new one. Delegation is by zones, not necessarily per label/server. Authoritative change does not immediately revoke caches.
2. No. Names map records, IP addresses route endpoints, ports demultiplex transports. User authentication and stable service identities require separate mechanisms.

Chapter 17 worked solutions • Distributed systems (continued)

OS17-E05 • Routes and packet pipelines

1. B begins∞;settlingD1 gives6;settlingC2 gives4;A–C–B costs 4. After C–B removal, A–D–B costs 6. Detection/convergence can still drop packets. Dijkstra assumes nonnegative fixed costs.
2. Eight ms/packet/hop gives(4+2−1) • 8=40 ms. Whole-message hops take32 ms each, total64. Headers require recalculation. This is the specified store-and-forward model.

OS17-E06 • IPv4, ARP, and a short frame

1. A ARPs192.0.2.1 forR_L. Frames areA→R_L thenR_R→B;both IP endpoints remain192.0.2.10→198.51.100.20. R resolves B on its right LAN, decrementsTTL, and updates checksum.
2. IP44 B needs2 B pad to46. Frame14+46+4=64 B. Add8 preamble/SFD and12 IFG byte-times to84;84 • 8/10^8=6.72 µs. IFG is not IP data. The model is untagged, no options,100 Mbit/s;MAC addresses are6 B.

OS17-E07 • Indistinguishable failure observations

1. B crashes before ping, or B remains alive with request/reply delayed beyond201 ms. Through200, A sees only send/timer, no reply, in both. Its history cannot distinguish them.
2. Earliest prior/latest next arrival gives.5+.1+.05=.65 s. Exceeding it contradicts some model premise or indicates a problem;invalid delay bounds invalidate no-false-suspicion claims. Asynchronous timeout proves neither crash nor a particular physical fault.

OS17-E08 • Fencing and retries

1. Partitioned A can still write, causing dual ownership. Correct coordination issues epochs;storage atomically installs43 and validates every write, rejecting42 before B operates. Independent increments in each partition do not provide unique authority.
2. Atomically persist counter++ and id17→result11. Retry returns 11. Counter update before ID permits duplicate increment on crash;ID before effect permits a missing increment. They need one recoverable transaction. At-most-once does not guarantee client knowledge or include external effects automatically.

Chapter 17 worked solutions • Distributed systems (continued)

OS17-E09 • DFS naming and cache cost

1. Change vol9 location, retain stable path/FID. Recreated file 2 needs a new generation against stale references. Invalidate/version location caches rather than retaining them forever.
2. 20 • 6+80 • .2=136 ms;20 • 4=80 KiB. Additional100 • 1 gives236 ms. Data hits need not eliminate network validation. Identity, location, validation cost, and semantics differ.

OS17-E10 • Consistency and partitions

1. Yes, v0 remains legal before revalidation. This is weaker than every read seeing completed writes. Periodic validation and synchronous invalidation are different contracts.
2. Before write completion, recall conflicting read rights or wait for controlled lease expiry;clients must stop using expired authority. Sending an unacknowledged invalidation without bounded leases cannot prove old-client cessation during partition. Callbacks do not change already-read application buffers.

OS17-E11 • NFSv3 verifier changes

1. Old UNSTABLE persistence is unproved. Retransmit unconfirmed D and obtain the required stable result with verifier checks. Normal same-instance successful COMMIT with matchingX permits releasing that batch's retransmission obligation.
2. Neither. Core calls identify handles/offsets without equivalent OPEN session state, while server caches/duplicate caches remain possible and WRITE has stability modes. Durability, visibility, and deduplication are distinct. This is RFC1813 NFSv3, not v2/v4.

OS17-E12 • Historical HDFS and failover capacity

1. Blocks64,64,22 MiB, total three. Replicated payload 450 MiB excludes metadata/checksums/allocation. NameNode manages namespace/locations;DataNodes carry client payload. Secondary NameNode checkpoints rather than acting as automatic hot standby.
2. Remaining100<150 leaves at least50/s unsustainable. Taking over does not preserve throughput/SLO without headroom, degradation, or limiting. Replica payload, physical allocation, and metadata/network reliability differ.

Chapter 17 worked solutions • Distributed systems (continued)

Verification code appendix: distributed_models.py

```
from fractions import Fraction as F
from heapq import heappush,heappop
def distance(graph,start):
 q=[(0,start)];d={start:0}
 while q:
  cost,u=heappop(q)
  if cost!=d[u]:continue
  for v,w in graph[u].items():
   c=cost+w
   if v not in d or c<d[v]:d[v]=c;heappush(q,(c,v))
 return d
def increment(state,requests,rid):
 # One atomic transaction in this pure model. Real storage must implement it.
 if rid in requests:return state,requests,requests[rid]
 state+=1;requests={**requests,rid:state};return state,requests,state

class NFSModel:
 def __init__(self):self.stable={};self.ram={};self.verifier=1
 def write_unstable(self,offset,data):
  self.ram[offset]=data
  return len(data),self.verifier
 def commit(self):
  self.stable.update(self.ram);self.ram.clear();return self.verifier
 def crash(self):
  self.ram.clear();self.verifier+=1

class FencedStore:
 def __init__(self):self.epoch=42;self.value=None
 def install(self,new_epoch):
  assert new_epoch>self.epoch
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 17 worked solutions • Distributed systems (continued)

```
  self.epoch=new_epoch
 def write(self,epoch,value):
  if epoch!=self.epoch:return False
  self.value=value;return True
if __name__=='__main__':
 g={'A':{'C':2,'D':1},'C':{'A':2,'B':2},'D':{'A':1,'B':5},'B':{'C':2,'D':5}}
 assert distance(g,'A')['B']==4;del g['C']['B'];del g['B']['C'];assert distance(g,'A')
['B']==6
 assert (4+2-1)*1040*8/1_000_000==.0416
 assert 14+20+20+1000+4==1058 and 1058+8+12==1078
 assert 14+max(46,20+20+4)+4==64
 assert 84*8/100_000_000==.00000672
 assert F(65536*8,1)/F(40,1000)==F(13107200,1)
 state,seen,result=increment(10,{},17);assert(state,result)==(11,11)
 state,seen,result=increment(state,seen,17);assert(state,result)==(11,11)
 assert F(20*6)+80*F(2,10)==136
 s=NFSModel();pending=b'D'*4096
 count,old=s.write_unstable(0,pending);assert count==4096 and s.stable=={}
 s.crash();new=s.commit();assert new!=old and s.stable=={}
 # Retain pending when generation mismatches; resend and commit new instance.
 count,new=s.write_unstable(0,pending);assert s.commit()==new
 assert s.stable[0]==pending
 s.crash();assert s.stable[0]==pending
 f=FencedStore();assert f.write(42,'old-owner-before-fence')
 f.install(43);assert not f.write(42,'stale') and f.write(43,'new-owner')
 assert f.value=='new-owner'
 print('route costs 4/6; pipeline41.6ms; frame1058/wire1078; shortframe64; BDP-
window13.1072Mbit/s')
 print('atomic retry id17: counter11 after duplicate; cache136ms; NFS crash loses RAM,
verifier mismatch/resend restores D; old epoch rejected')
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 17 worked solutions • Distributed systems (continued)

Verification code appendix: rpc_retry.py

```
"""Model retry after a lost response in one serial, non-crashing server.

The in-memory dictionary is not durable and there is no concurrent access.
This example does not prove exactly-once effects across crashes.

Run:
    python rpc_retry.py
"""


class Server:
    def __init__(self) -> None:
        self.commits: dict[str, str] = {}
        self.executions = 0

    def create_once(self, request_id: str, name: str) -> str:
        if request_id in self.commits:
            return self.commits[request_id]
        self.executions += 1
        result = f"object:{name}"
        self.commits[request_id] = result
        return result


def main() -> None:
    server = Server()
    first = server.create_once("req-17", "notes")
    retry = server.create_once("req-17", "notes")
    assert first == retry
    assert server.executions == 1
    print("result", retry, "executions", server.executions)


if __name__ == "__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study